IA et Génie Logiciel MLOps
TP 1 : Modèle ML & API

Objectif pédagogique du TP

L'objectif de ce premier TP n'est pas de créer le modèle le plus performant du monde, mais d'apprendre à l'**industrialiser**. Vous allez passer du statut de "Data Scientist" (qui explore des données dans un notebook) à celui de "Machine Learning Engineer" (qui package un modèle, s'assure de sa reproductibilité et l'expose pour qu'il soit utilisable par d'autres applications).

Important : vous **choisissez vous-même** le jeu de données et le type de problème (régression ou classification). L'évaluation se fera via une grille **indépendante du dataset** : nous évaluerons la qualité de l'implémentation, la reproductibilité, les tests et la documentation.

1 Le point de départ : Choix et compréhension des données

En MLOps, on dit souvent "Garbage in, Garbage out". Un bon pipeline commence par des données claires et bien définies.

1.1 Exigences minimales

Vous devez choisir un dataset qui respecte ces critères pour garantir que le TP reste réalisable en temps limité :

- **Taille raisonnable :** idéalement $<$ **200 MB** pour un entraînement de votre modèle de base en $<$ **2 minutes**.
- **Données structurées :** format CSV, Parquet ou JSON tabulaire (pas de vision ou d'audio pour ce TP).
- **Une variable cible claire :** il doit s'agir d'un problème de **classification** (binaire ou multi-classes) ou de **régression**.
- **Propriété intellectuelle :** La licence ou les conditions d'utilisation doivent être mentionnées dans votre fichier `README.md`.

1.2 Où trouver votre bonheur ?

- UCI Machine Learning Repository : https://archive.ics.uci.edu/
- OpenML : https://www.openml.org/
- Hugging Face Datasets : https://huggingface.co/datasets
- Portails Open Data : https://www.data.gouv.fr/
- Kaggle : https://www.kaggle.com/datasets

1.3 Justification obligatoire (à inclure dans le README)

Prenez du recul sur vos données. Expliquez en **5 à 10 lignes** :

- **Le contexte :** Pourquoi ce dataset est-il intéressant ?
- **La tâche :** Régression ou classification ?
- **L'anatomie des données :** Nombre de features, types de variables, et taille globale.
- **Les défis anticipés :** Y a-t-il des valeurs manquantes, un déséquilibre de classes, des outliers ?

2 Le cœur du réacteur : Préparation, Entraînement et Tests

Un modèle d'IA n'a de valeur que s'il est reproductible. C'est ici que l'ingénierie logicielle entre en jeu.

2.1 Isolation de l'environnement

Conseil

Ne travaillez jamais dans l'environnement Python global de votre machine. Un environnement virtuel garantit que votre projet fonctionnera à l'identique sur l'ordinateur de votre collègue (ou de votre correcteur !).

Étape 1 : Créer un environnement virtuel

```
python -m venv .venv

# Sur Linux/Mac :
source .venv/bin/activate

# Sur Windows (PowerShell) :
# .\.venv\Scripts\Activate.ps1
```

Étape 2 : Installer les dépendances (via un fichier `requirements.txt`)

```
pip install --upgrade pip
pip install -r requirements.txt
```

(Dépendances suggérées : scikit-learn, pandas, numpy, joblib, fastapi, uvicorn, pydantic, pytest, httpx).

2.2 Chargement et séparation des données

Vous devez fournir **une méthode reproductible** pour obtenir les données (un script de téléchargement, un dataset intégré, ou un dossier `data/` bien organisé).

Le fractionnement (Split) :

- Séparez vos données en 3 : **Train 70% / Validation 15% / Test 15%**.
- *Classification :* Pensez à utiliser `stratify=y` pour conserver la proportion de vos classes.
- **Traceabilité (Obligatoire) :** Consignez dans un fichier `run_info.json` : le nom du dataset, sa forme (lignes/colonnes), la cible, la fraction du split et le `random_state`.

2.3 Construction du Pipeline Scikit-Learn

Ne séparez pas le prétraitement (preprocessing) de votre modèle. Utilisez un **pipeline scikit-learn** (`ColumnTransformer`, etc.).
Pourquoi ? Cela rend l'inférence beaucoup plus simple. Le pipeline complet est sérialisé en une seule fois, évitant les fuites de données (data leakage) et les erreurs lors du passage en production.

2.4 Suivi des performances (Métriques)

Choisissez au moins 2 métriques adaptées à votre tâche (ex : Accuracy et F1-score pour la classification ; RMSE et MAE pour la régression).
Obligatoire : Stockez ces résultats dans un fichier `metrics.json` (noms, valeurs, horodatage, hyperparamètres clés).

2.5 Sauvegarde des artefacts

L'entraînement produit des "artefacts" (fichiers générés) qui seront consommés par l'API. Dans `src/mlops_tp/artifacts/`, générez :

1. `model.joblib` : votre pipeline complet entraîné.
2. `metrics.json` : l'historique de votre entraînement.
3. `feature_schema.json` : un dictionnaire décrivant les variables attendues.

2.6 Tests Automatisés (Pytest)

Le code ML doit être testé comme n'importe quel code logiciel. Créez au minimum :

— `test_training.py` : Vérifie que l'entraînement s'exécute de bout en bout et génère bien le fichier `model.joblib`.
— `test_inference.py` : Vérifie que la fonction `predict` renvoie bien un scalaire (régression) ou une classe connue (classification), et que `predict_proba` (si applicable) renvoie des valeurs entre 0 et 1.

3 L'ouverture au monde : Servir votre modèle via une API REST

Votre modèle est prêt. Maintenant, il faut permettre à d'autres applications (un site web, une application mobile) de l'interroger.

Vous allez exposer votre modèle via une API REST utilisant le framework **FastAPI**.

3.1 Les "Endpoints" (Routes de l'API)

Votre API doit exposer les routes suivantes, qui doivent être documentées :

— `GET /health` : Permet de vérifier si l'API est "vivante" et prête à répondre.
— `GET /metadata` : Renvoie la version du modèle, le type de tâche, et les variables attendues.
— `POST /predict` : Reçoit les nouvelles données et renvoie la prédiction.

3.2 Le contrat d'interface (JSON)

Pour que ce TP soit générique quel que soit votre dataset, on impose cette structure de requête pour `POST /predict` :

```
{
  "features": {
    "surface": 52.0,
    "rooms": 2,
    "city": "Clermont-Ferrand"
  }
}
```

Exemple de réponse attendue (Classification) :

```
{
  "prediction": "yes",
  "task": "classification",
  "proba": {"yes": 0.82, "no": 0.18},
  "model_version": "0.1.0",
  "latency_ms": 2.6
}
```

3.3 Robustesse et Validation

Conseil

Ne faites jamais confiance aux données envoyées par l'utilisateur !

- Si une variable manque ou si son type est incorrect (ex : du texte au lieu d'un nombre), l'API doit renvoyer une erreur 422 (Unprocessable Entity) avec un message clair.
- **Performance :** Chargez votre fichier `model.joblib` **une seule fois** au démarrage de l'application (au lancement du serveur), et non à chaque fois qu'un utilisateur fait une requête !

3.4 Tester son API

Lancez votre serveur avec : `uvicorn mlops_tp.api:app -host 127.0.0.1 -port 8000`
Allez sur http://127.0.0.1:8000/docs pour tester vos routes grâce à l'interface Swagger générée automatiquement par FastAPI.

Créez ensuite le fichier `tests/test_api.py` pour automatiser ces vérifications (les codes HTTP 200 pour le succès, 422 pour les erreurs).

4 Organisation recommandée du dépôt

Un bon projet MLOps est avant tout un projet bien rangé :

```
TP_MLOps/
  README.md
  requirements.txt
  src/
    mlops_tp/
      __init__.py
      config.py
      train.py
      inference.py
```

```
    api.py
    schemas.py
    artifacts/
      model.joblib
      metrics.json
      feature_schema.json
      run_info.json
tests/
  test_training.py
  test_inference.py
  test_api.py
```